

Scraping without suspicion

Scraping websites isn't as easy as using a simple tool, these days websites fight aggressive scraping methods

Mithun Mohandas | mithun@digit.in

If you're ever worked on a project that needed a lot of data, then one of your go to destinations for obtaining data would have been the internet. With millions of websites out there hosting petabytes of data, where else would you have to look if you wanted tonnes of data for free? Getting this data wasn't so hard either, there

server resources consumed by the website and even affect the experience of genuine visitors if you send too many requests to the server. Which is why rate limiters are common with many CMS these days.

On one end, scraping publicly available data shouldn't be a problem, and enabling this goes towards maintaining an open internet. On the other hand, wasting server resources and preventing genuine visitors to accessing the same data goes against the same philosophy of openness. So those in need of data need to strike a balance with the number of requests that they can send so that they can scrape data as quickly as possible without setting off the rate limiters and other protection mechanisms. Let's take a look at how this can be done.

WHY DO PEOPLE SCRAPE

There are a number of common use cases wherein you can use scraping

aggressive scraping in order to be competitive. Ideally, if the website being scraped has an API then that should be leveraged instead of scraping. For example, Amazon price trackers tend to use the Amazon API rather than manually scraping product listings every few minutes.

But what do you do when the website in question does not have an API or if it does but the API endpoint has not been made publicly accessible? You slow down and emulate the browsing habits of a normal user. It's extremely slow but with proxies and varying request patterns can easily allow you to speed up your data collection process. We've previously covered using Scrapy to scrape websites. Think of this article as a continuation along that one with a focus on ensuring that you don't get locked out.

MODIFY YOUR USER-AGENT

Whenever you send a request to a web server, the data packets will always have a header that indicates the User-Agent of the client that is making the request. User-Agents are unique to their respective browsers so they make it extremely easy to identify whether the client is a human or a bot.

However, we've all known that User-Agents can be easily spoofed. Which is why websites now look at multiple aspects to identify bots. One of the methods is to query the client for something that's unique to their feature set. For example, if a certain HTML code is only compatible with Internet Explorer, then observing if that code is rendered can easily tell the site whether the client is genuinely making a request or if it's a bot with a spoofed User-Agent.



Scrapy

have been a lot of automated tools over the years which any Tom, Dick and Harry can simply power up and point to a website. Previously, websites wouldn't have put up a fight since all the information put up on a website was for public consumption anyway. However, scrapers can often spike the

tools. Let's say you're trying to create an archive of a certain section of a website. Using a scraping tool is much more effective rather than manually opening web pages and save them as HTML. Services which aggregate data such as product listings, news articles, leads, etc. have to perform ag-